

El Corazon

Ecosysteme de livraison de repas — 3 apps Flutter + Supabase

Rapport global d'analyse

Architecture, securite, qualite du code, dette technique & feuille de route

430

fichiers Dart

~176 000

lignes de code

3

applications

24

fichiers > 1000 lignes

Genere par analyse statique du depot — 22 juillet 2026

Confidentiel — usage interne

SYNTHESE EXECUTIVE

1. Vue d'ensemble

El Corazon est un ecosysteme complet de livraison de repas compose de trois applications Flutter interconnectees via une base de donnees Supabase commune : une application **client**, une application **livreur** et un **panneau d'administration**. Le depot represente un monorepo d'environ **430 fichiers Dart** et **176 000 lignes de code**.

Le produit est fonctionnellement riche et avance (85 a 95 % de completion selon les apps), mais l'analyse statique revele une **dette technique importante** : duplication massive entre apps, classes surdimensionnees, gestion d'erreurs faible, et surtout une **couverture de tests quasi nulle** sans aucune integration continue. Un plan de refactoring en 6 phases (16 a 24 jours) a ete etabli et les correctifs de securite critiques ont deja ete appliques.

Tableau de bord — scores par domaine

Domaine	Score	Etat	Priorite
Fonctionnel / Produit	8 / 10	Riche, 85-95% complet	—
Securite	6 / 10	Secrets corriges, rotation cles a faire	HAUTE
Architecture	4 / 10	God classes, duplication, couplage	HAUTE
Qualite du code	5 / 10	Erreurs avalees, casts non surs	MOYENNE
State management	4 / 10	Provider + Riverpod non initialise	HAUTE
Tests	1 / 10	~0% couverture, 1 test qui echoue	CRITIQUE
CI / CD	1 / 10	Aucun pipeline	CRITIQUE

Note : les scores decoulent de l'audit statique du 21 juillet 2026 ; ils sont indicatifs et destines a prioriser le travail, non a juger le produit fini.

Recommandations prioritaires

- **Immediat (critique)** : roter manuellement les cles Supabase et Google Maps precedemment exposees ; mettre en place une CI minimale (flutter analyze + tests).
- **Court terme** : creer un package partage **elcora_shared** pour eliminer ~40 fichiers dupliques ; deplacer PayDunya et la generation de tokens Agora vers un backend serveur.
- **Moyen terme** : decouper les classes de plus de 1000 lignes, structurer la gestion d'erreurs, et etabli une premiere couverture de tests.

ARCHITECTURE

2. Architecture de l'écosystème

L'écosystème suit une architecture modulaire : trois clients Flutter distincts partagent une infrastructure backend unifiée (Supabase). Un dossier **backend/** est prévu pour un futur serveur (actuellement il ne contient qu'un `.env.example` — le backend Laravel reste à initialiser).

Couche	Composants
Clients Flutter	elcora_fast (client) • elcora_dely (livreur) • admin (back-office)
Infrastructure	Supabase : PostgreSQL, Auth, Storage, Realtime
Services externes	Google Maps, Firebase Cloud Messaging, Agora RTC, PayDunya
Backend (a venir)	Laravel — dossier backend/ vide (env.example uniquement)

Les trois applications

App / dossier	Role	Fichiers	Lignes	SDK	Compl.
elcora_fast (El Corazon fastfood)	Client final : catalogue, panier, paiement, suivi livraison, gamification	194	78 482	^3.5.0	~85%
admin (El Corazon admin)	Back-office : dashboard, commandes, menu, livreurs, analytics	154	64 987	>=3.5 <4	~95%
elcora_dely (El corazon dely)	Livreur : courses, navigation GPS, gains, chat/appels, notifications	82	33 035	^3.9.2	~90%
TOTAL	Monorepo	430	176 504	—	—

Pile technique

- **Framework** : Flutter / Dart, avec Provider (les 3 apps) et Riverpod partiellement installé dans le client.
- **Données** : Supabase (PostgreSQL, Auth, Realtime, Storage) ; stockage local via sqflite, shared_preferences, flutter_secure_storage.
- **Cartes & position** : google_maps_flutter, geolocator.
- **Communication** : Agora RTC (audio/video), Firebase Cloud Messaging (notifications), speech_to_text (commandes vocales, livreur).
- **Paieement** : PayDunya (Mobile Money Afrique de l'Ouest) + wallet interne.
- **Admin** : fl_chart & syncfusion pour les graphiques, pdf/printing pour les rapports.

SECURITE

3. Securite

L'audit a identifie plusieurs expositions de secrets, dont la plupart ont deja ete corrigees dans le code. Il reste une action manuelle essentielle : **la rotation des cles precedemment exposees** dans leurs dashboards respectifs, car elles ont pu etre lues tant qu'elles etaient presentes dans le depot.

Constat	Detail	Etat
Secrets hardcodes	Cle Google Maps + URL/anon Supabase en const dans api_config.dart (dely)	Corrige (dotenv)
.env commite	El corazon dely/.env suivi par git (git rm --cached applique)	Corrige
Cles PayDunya placeholder	'YOUR_PAYDUNYA_MASTER_KEY' — paiement non fonctionnel en prod	Partiel
Cles Agora placeholder	App ID cote client, pas de token RTC serveur	Partiel
Rotation des cles	Cles Supabase & Google Maps exposees a invalider manuellement	A FAIRE
Config split-brain (dely)	2 projets Supabase references (.env vs api_config.dart)	A verifier

Recommandations serveur

- Deplacer la logique PayDunya cote serveur (cles secretees jamais dans le binaire client) et implementer les webhooks.
- Generer les tokens Agora RTC cote serveur a partir de l'App Certificate (jamais expose au client).
- Faire transiter le geocoding par le backend pour maitriser les quotas Google Maps.
- Remplacer les cles live des fichiers .env.example par des placeholders et verifier le .gitignore des 3 apps.

QUALITE & DETTE TECHNIQUE

4. Qualite du code et dette technique

4.1 Metriques statiques

Metrique	Valeur	Lecture
Fichiers Dart	430	—
Lignes de code	~176 000	—
Fichiers > 1000 lignes	24	God classes a decouper
Fichiers > 2000 lignes	5	Cas les plus critiques
catch (e) sans stack trace	1 303	Erreurs avalees
debugPrint bruts	2 133	Pas de logging structure
as Type (cast non sur)	877	Risque de crash runtime
parse() non securise	33	A remplacer par tryParse
Singletons manuels	106	Contournent l'arbre Provider
Fichiers dupliques entre apps	~40	Divergences silencieuses
Variantes enhanced_/advanced_/modern_	22	Doublons a arbitrer

4.2 Les fichiers les plus volumineux

Les 12 plus gros fichiers concentrent l'essentiel de la complexite. Ils melangent ecran, logique metier et widgets, et devraient etre decoupees en composants.

#	Fichier	App	Lignes
1	advanced_order_management_screen.dart	admin	3 016
2	group_order_screen.dart	fastfood	2 867
3	cake_order_screen.dart	fastfood	2 799
4	order_management_screen.dart	admin	2 655
5	database_service.dart	fastfood	2 255
6	enhanced_admin_dashboard.dart	admin	2 002
7	gamification_management_screen.dart	admin	1 933
8	database_service.dart	dely	1 683
9	delivery_tracking_screen.dart	fastfood	1 669
10	social_service.dart	fastfood	1 382
11	app_service.dart	fastfood	1 339
12	app_service.dart	dely	1 333

4.3 Duplication entre applications

Environ 40 fichiers sont copies-colles entre les apps avec des divergences non intentionnelles, sources de bugs subtils :

- **price_formatter.dart** : fastfood affiche "1.000 CFA" (point), admin affiche "1 000 CFA" (espace).
- **database_service.dart** : 3 versions (72 methodes en fastfood, signatures differentes en dely, 7 methodes en admin).
- **cart_service.dart** : 3 versions + un enhanced_cart_service.dart = 4 implementations.
- **Modeles** (user, order, cart_item, address, menu_item...) dupliques avec des champs non alignes.
- **paydunya_service.dart** : URL de base hardcodee differemment selon l'app.

TESTS & INTEGRATION CONTINUE

5. Tests & CI/CD

C'est le point le plus faible du projet. La couverture de tests est **quasi nulle** et il n'existe **aucun pipeline d'integration continue**. Le seul test present (admin) echoue faute d'initialisation de dotenv/Supabase.

Application	Repertoire test/	Tests	Couverture
elcora_fast	Absent	0	~0%
elcora_dely	Absent	0	~0%
admin	Present	1 (echoue)	~0%

Etat CI/CD

- Aucun fichier .github/workflows/ — pas de CI, pas de CD.
- analysis_options.yaml strict cote fastfood mais non applique (linter non enforce).
- Aucun test unitaire, d'integration ou de widget sur les parcours critiques (checkout, login, panier).

Plan de mise en place

- Tests unitaires d'abord sur le futur package partage (formatters, validators, modeles).
- Tests de services avec Supabase mocke (Mockito).
- Widget tests sur checkout, login et panier.
- Pipeline GitHub Actions : flutter analyze + flutter test bloquants a la fusion.
- Harmoniser analysis_options.yaml sur les 3 apps (aligner sur la config stricte de fastfood).

FEUILLE DE ROUTE

6. Plan de refactoring en 6 phases

Un plan detaille a ete etabli (REFACTORING_PLAN.md / docs/REFACTORING_AUDIT.md). Estimation globale : **16 a 24 jours** de travail, executes dans l'ordre de priorite ci-dessous.

Phase	Objet	Duree	Priorite
1	Securite critique — roter les clefs, unifier la config secrets	1-2 j	CRITIQUE
2	Package partage elcora_shared — supprimer ~40 doublons	3-5 j	HAUTE
3	God classes — decouper les fichiers > 1000 lignes	5-7 j	MOYENNE
4	Qualite — erreurs structurees, casts surs, tryParse	3-4 j	MOYENNE
5	Tests & CI — couverture minimale + pipeline	3-4 j	IMPORTANT
6	Polish — assets, entypoints, .env.example, docs	1-2 j	FAIBLE

Conclusion

El Corazon est un produit ambitieux et deja largement fonctionnel, porte par une base de code consequente. Sa valeur est reelle, mais sa **maintenabilite est menacee** par la duplication, les classes surdimensionnees et surtout l'absence de tests et de CI. Les correctifs de securite critiques etant deja engages, la priorite suivante est claire : **mutualiser le code partage et instaurer un filet de securite automatise** (tests + pipeline) avant de poursuivre le decoupage architectural. En suivant le plan en 6 phases, le projet peut retrouver une trajectoire de developpement saine en 3 a 5 semaines.

Sources : README.md, DOCUMENTATION_GLOBALE.md, REFACTORING_PLAN.md, docs/REFACTORING_AUDIT.md, et analyse statique en direct du depot (comptages de fichiers, lignes, occurrences catch/debugPrint verifiees le 22/07/2026).